Polymorphism in C#

1. What is Polymorphism?

Polymorphism means **"many forms."**

In object-oriented programming, polymorphism allows **one method, operator, or interface to behave differently depending on the context or object**.

👉 In simple terms:
The same method name can perform different actions.

Real-world Example

Think about the **"Draw"** action.

- A **Circle** draws a circle
- A **Rectangle** draws a rectangle
- A **Triangle** draws a triangle

Same action (**Draw**) → different implementation.

Types of Polymorphism in C#

C# supports **two types of polymorphism**.

Type	Also Called	When it happens
Compile Time Polymorphism	Static Polymorphism	Decided during compilation
Runtime Polymorphism	Dynamic Polymorphism	Decided during execution

---

1. Compile-Time Polymorphism (Method Overloading)

This occurs when **multiple methods have the same name but different parameters**.

The compiler decides **which method to call**.

Example: Method Overloading

class Calculator

```
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }
}
```

Using the Class

```
Calculator calc = new Calculator();

Console.WriteLine(calc.Add(5, 10));       // Calls first method
Console.WriteLine(calc.Add(5, 10, 15));   // Calls second method
Console.WriteLine(calc.Add(5.5, 2.3));    // Calls third method
```

Output

```
15
30
7.8
```

Key Rules for Method Overloading

Methods must differ by:

- Number of parameters
- Type of parameters
- Order of parameters

❌ This is **NOT allowed**:

int Add(int a, int b)

double Add(int a, int b)

Return type alone **cannot differentiate methods**.

---

2. Runtime Polymorphism (Method Overriding)

Runtime polymorphism occurs through **inheritance** using:

- virtual
- override

The **method call is determined at runtime**.

Example: Method Overriding

Base Class

```
class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal makes a sound");
    }
}
```

Derived Class

```
class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Another Derived Class

```
class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Cat meows");
    }
}
```

Using Runtime Polymorphism

```
Animal a;

a = new Dog();
a.Speak();

a = new Cat();
a.Speak();
```

Output

Dog barks

Cat meows

Here:

- The **reference type = Animal**
- The **object type changes**

At runtime C# decides **which method to call**.

Why Polymorphism is Important

Polymorphism helps in:

1. Code Reusability

We can write generic code that works with many object types.

2. Flexibility

Programs become easier to extend.

3. Maintainability

Less duplicate code.

4. Loose Coupling

Components interact through common interfaces.

Real Project Example (Payment System)

Imagine an **e-commerce payment system**.

```
class Payment
{
    public virtual void Pay()
    {
        Console.WriteLine("Processing payment");
    }
}
```

Credit Card Payment

```
class CreditCard : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Paid using Credit Card");
    }
}
```

UPI Payment

```
class UPI : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Paid using UPI");
    }
}
```

Execution

```
Payment p;

p = new CreditCard();
p.Pay();

p = new UPI();
p.Pay();
```

Output

Paid using Credit Card

Paid using UPI

Same **Pay() method** → different implementations.

Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Polymorphism Type	Compile Time	Runtime
Inheritance Required	No	Yes
Keyword	None	virtual, override
Method Name	Same	Same
Parameters	Must be different	Must be same

Key Keywords Used in Polymorphism

Keyword	Purpose
virtual	Allows a method to be overridden
override	Overrides base class method
new	Hides base class method
abstract	Forces derived class to implement method

Example:

```
public virtual void Display()

public override void Display()
```

Summary

Polymorphism allows **one interface to represent multiple behaviors**.

Two types in C#:

1️⃣ **Compile Time Polymorphism**

- Method Overloading
- Decided at compile time

2️⃣ **Runtime Polymorphism**

- Method Overriding
- Uses inheritance
- Decided at runtime